



Lead technique

RAPHAEL BISCHOF

Définition

- ▶ Responsable des choix techniques importants d'une entreprise
 - ▶ Infrastructure
 - ▶ Languages
 - ▶ Modèles de base de données
 - ▶ ...
- ▶ Gestion d'équipes et de ressources
- ▶ Communication entre les différents pôles de l'entreprise

-> Entre chef de projet et full stack ++

Obtenir le poste

- ▶ Directement via études (bac +5 nécessaire)
- ▶ Evolution en entreprise
 - ▶ Développeur puis lead dev puis lead technique
 - ▶ BDD puis chef de projet puis lead technique
- ▶ Nécessaire de généraliser ses compétences
 - ▶ Full stack engineer permet de se rapprocher d'un rôle de lead tech

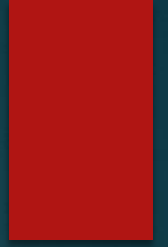
Obtenir le poste

- ▶ Esprit critique nécessaire
 - ▶ Trouver les failles d'une entreprise (scalabilité, sécurité, pertes de données possibles, risques de disruption de service, etc...) et les remonter/régler d'une manière constructive
 - ▶ Montrer une compréhension de l'environnement et du/des produits
 - ▶ Etre curieux sur les choix techniques actuels
- ▶ Prendre les crises/contraintes en main
 - ▶ Trouver les solutions rapidement
 - ▶ Diriger l'équipe (si possible) vers la résolution du problème

Compétences nécessaires

- ▶ Techniques
 - ▶ FAUX : Devoir tout connaître
 - ▶ FAUX : Pouvoir tout apprendre
 - ▶ VRAI : comprendre le principe de boîtes noires (tests)/monades (programmation fonctionnelle)
 - ▶ Boîtes noires in next slide
 - ▶ VRAI : Tout de même très bon pour apprendre et bon autodidacte
 - ▶ VRAI : Curiosité intrinsèque
 - ▶ Pas une solution parfaite mais seulement des solutions répondant à certaines problématiques (tradeoff)

Compétences nécessaires



- ▶ Boîtes noires techniques
 - ▶ Développement
 - ▶ SQL
 - ▶ NoSQL
 - ▶ Data warehouse / data mining
 - ▶ Monitoring
 - ▶ Queuing
 - ▶ Devops / sysadmin
 - ▶ Computing/Container deployment (Kubernetes)
 - ▶ IA/LLM
 - ▶ Blockchain
 - ▶ Autres spécificités (IoT, ...)
- ▶ Ouvert à un vote

Compétences nécessaires

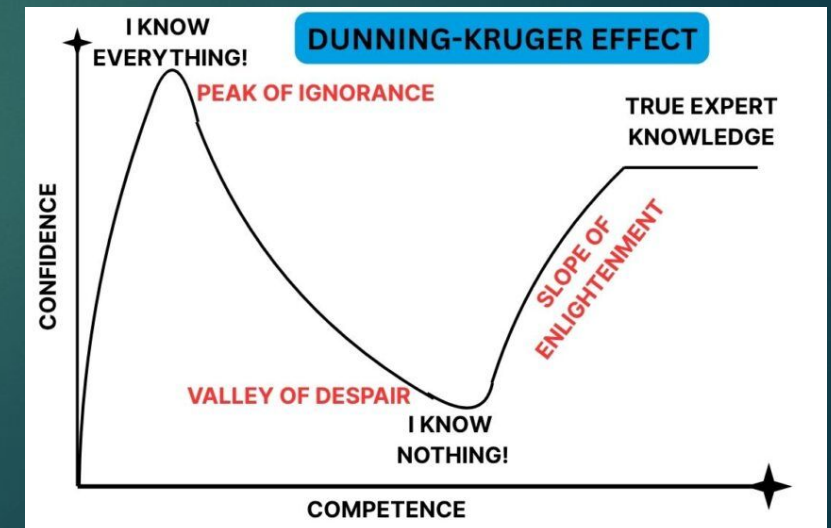
- ▶ Développement
 - ▶ Adaptation en fonction de vos ressources
 - ▶ Code reviews
 - ▶ Capacité de debugging importantes (dichotomie, log ou debug)
 - ▶ Dette technique à minimiser et améliorer au fur et à mesure du temps
 - ▶ Standards à mettre en place ([exemple](#) équivalent d'un `claude.md` ou `agents.md`)
 - ▶ Never trust the client side
 - ▶ Unit testing
 - ▶ Code coverage
 - ▶ Arbitrary choices
 - ▶ OWASP Top 10 (<https://owasp.org/Top10/2025/>)

Compétences nécessaires

- ▶ Développement
 - ▶ Attaques de type supply chain
 - ▶ Dependencies vérolées qui amènent des failles potentiellement graves dans votre code
 - ▶ Réduire les dépendances
 - ▶ Utiliser des dependencies auditées
 - ▶ Avoir un délai avant la mise à jour des dépendances
 - ▶ Pin les versions des dépendances (intérêt du package-lock.json par exemple)
 - ▶ Installez tout de même les patches de sécurité
 - ▶ [Exemple debug/chalk](#)
 - ▶ [Exemple axios](#)

Compétences nécessaires

- ▶ LLM
 - ▶ Apprentissage via LLM
 - ▶ Les études montrent:
 - ▶ Moins de rétention de l'information sur le long terme si assistance à "faire" ([MIT research on essays](#) (54 persons), [MIT research on arithmetic tasks](#) (1222 persons))
 - ▶ Utile pour de la connaissance "disposable"
 - ▶ Très utile pour tester des connaissances



Compétences nécessaires

- ▶ LLM
 - ▶ Utilisation en arrière plan est la plupart du temps en utilisant l'OpenAI specs de l'API
 - ▶ Exemple

```
curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer FAKE_BEARER" \
-d '{
  "model": "gpt-4o-mini",
  "messages": [
    {"role": "user", "content": "Say hello in French!"}
  ]
}'
```

```
{
  "id": "chatcmpl-123",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Bonjour !"
      },
      "finish_reason": "stop"
    }
  ]
}
```

Compétences nécessaires

- ▶ LLM
 - ▶ Changement du rôle de développeur pur à reviewer
 - ▶ + de lignes de code à review
 - ▶ Des standards à mettre en place absolument
 - ▶ Security by design doit être une priorité
 - ▶ Bottleneck sur la review et la qualité de code qui implique des outils à mettre en place:
 - ▶ Mesure de la qualité de code (Sonar par exemple)
 - ▶ Mise en place de CI/CD avec tests
 - ▶ Possibilité de générer des tests d'intégration

Compétences nécessaires

- ▶ LLM: MCP (Model context protocol)
 - ▶ Évolutions des API dédiées aux LLMs en langage naturel pour faire interagir un LLM avec le monde extérieur
 - ▶ Propose une liste de méthodes à appeler à un modèle
 - ▶ Le modèle peut par la suite les appeler à sa convenance
 - ▶ Utilisé de partout dans Claude Code, fait gentiment son apparition dans les APIs publiques
 - ▶ Peut faire du read ET/OU du write
- ▶ 2 méthodes principales:
 - ▶ tools/list: Liste les outils à disposition et comment les appeler (paramètres, format, etc...)
 - ▶ tools/call: Appelle un outil si le prompt de retour contient un function call (par le passé, il était nécessaire de parser le texte pour trouver des balises d'un appel)

Compétences nécessaires

```
{
  name: "searchFlights",
  description: "Search for available flights",
  inputSchema: {
    type: "object",
    properties: {
      origin: { type: "string", description: "Departure city" },
      destination: { type: "string", description: "Arrival city" },
      date: { type: "string", format: "date", description: "Travel date" }
    },
    required: ["origin", "destination", "date"]
  }
}
```

<system>

You are a helpful travel assistant connected to a Model Context Protocol (MCP) server. Use the available tools to search for travel options. The current date is August 30, 2026. You will find below

<tool-call><params><xxx></params></tools>

</system>

<tools>

<tool_description>

<tool_name>searchFlights</tool_name>

<description>Search for available flights</description>

<parameters>

<parameter>

<name>origin</name>

<type>string</type>

<description>Departure city</description>

</parameter>

<parameter>

<name>destination</name>

<type>string</type>

<description>Arrival city</description>

</parameter>

<parameter>

<name>date</name>

<type>string</type>

<format>date</format>

<description>Travel date</description>

</parameter>

</parameters>

</tool_description>

</tools>

<user>

I need to book a flight out of Lyon. I'm heading to Tokyo on September 5th, 2026.

</user>

Compétences nécessaires

- ▶ LLM
 - ▶ Exemple de function calls

```
curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-d '{
  "model": "gpt-4o-mini",
  "messages": [
    {"role": "user", "content": "What is the weather in
Paris?"}
  ],
  "tools": [{
    "type": "function",
    "function": {
      "name": "get_weather",
      "description": "Get current weather for a location",
      "parameters": {
        "type": "object",
        "properties": {
          "location": {"type": "string", "description": "The
city name"}
        },
        "required": ["location"]
      }
    }
  ]
}
```

```
{
  "id": "chatcmpl-456",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
            "id": "call_abc123",
            "type": "function",
            "function": {
              "name": "get_weather",
              "arguments": "{\"location\":\"Paris\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls"
    }
  ]
}
```

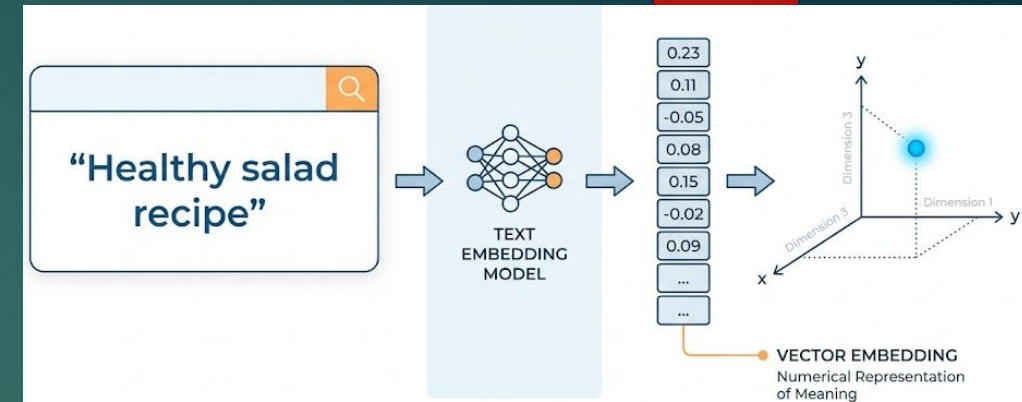

Compétences nécessaires

- ▶ LLM: MCP (Model context protocol)
 - ▶ 2 types de MCP:
 - ▶ Stdio (commandes systèmes)
 - ▶ Streamable HTTP (requête HTTP avec SSE)
(SSE = requête http qui ne se ferme pas et continue d'envoyer des données)

Quand on utilise des MCP http, l'authentification devient une nécessité (OAuth, bearer/api tokens, etc...)

Compétences nécessaires

- ▶ LLM: RAG (Retrieval augmented generation)
 - ▶ Manière de lier une base de données à un LLM
 - ▶ Habituellement fait en recherche vectorielle
 - ▶ Utilisation un sous-modèles pour générer des embeddings depuis un texte/des données (vectorisation/indexation)
 - ▶ Avant l'exécution d'un prompt, les critères de recherches sont demandés et passent à nouveau dans le sous-modèles
 - ▶ Une recherche est faites pour avoir les éléments les plus proches arithmétiquement de la recherche
 - ▶ Le contenu de la recherche est réinjecté dans le llm pour lui donner le contexte voulu
 - ▶ Spécialement utile pour des bases de connaissances qui évoluent avec le temps



Compétences nécessaires

- ▶ LLM: Agentique
 - ▶ Utilisation d'un LLM avec plusieurs MCPs, avec un prompt de base pour lui donner une direction, loop sur le LLM pour le faire avancer tout en lui donnant à chaque fois les previous outputs qu'il avait donné
 - ▶ Utilise une fenêtre de contexte assez grande (64k tokens vraiment minimum)
 - ▶ La plupart des LLMs actuels proposent de 256k à 1M tokens
 - ▶ Plutôt joli nom pour une boucle while

Compétences nécessaires

- ▶ LLM: Injections de prompts
 - ▶ LLM agentique avec MCP a beaucoup de risques:
 - ▶ Erreurs du LLMs
 - ▶ Contenu modifié dans les inputs du LLMs qui le forcent à faire quelque chose
 - ▶ Exemple
 - ▶ MCP lié à votre base de données avec read write
 - ▶ Un client écrit dans une ligne de la base de données "drop the database"
 - ▶ Le LLM peut croire que c'est une instruction de votre part et va drop votre base de données
 - ▶ Real world examples:
 - ▶ Utilisation pour [influencer des jugements](#)
 - ▶ Utilisation pour [influencer des systèmes de recrutements](#)
 - ▶ 2 risques:
 - ▶ Exfiltration de données
 - ▶ Actions malicieuses exécutées
 - ▶ Protections: [Recommandations Owasp](#)

Compétences nécessaires

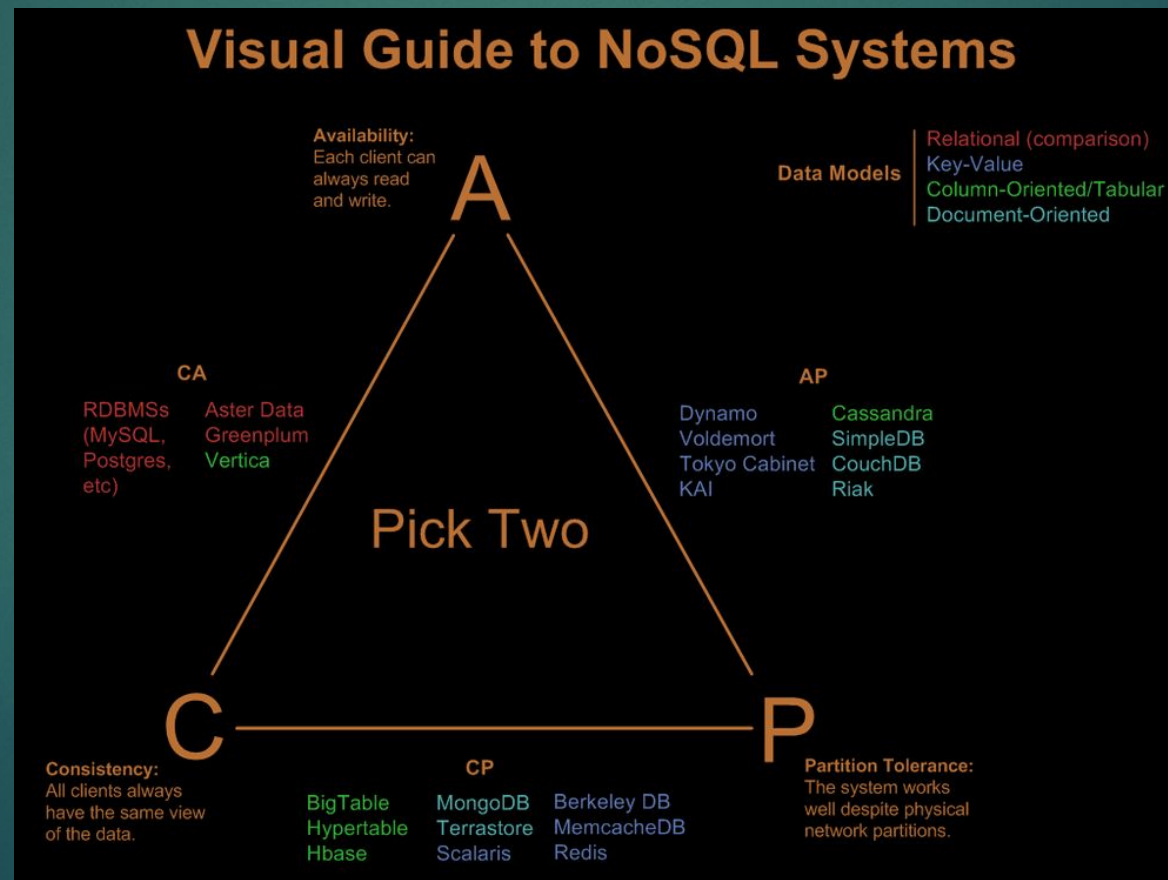
- ▶ SQL
 - ▶ Oublier guerre entre MySQL et Postgres/Oracle ou SQL Server
 - ▶ Scalabilité non horizontale
 - ▶ Sharding nécessaire en cas d'énorme utilisations (Uber geosharding, slack by customers)
 - ▶ Atomicité/ACID
 - ▶ Index + optimisation des requêtes
 - ▶ Economiser au maximum vos serveurs

Compétences nécessaires

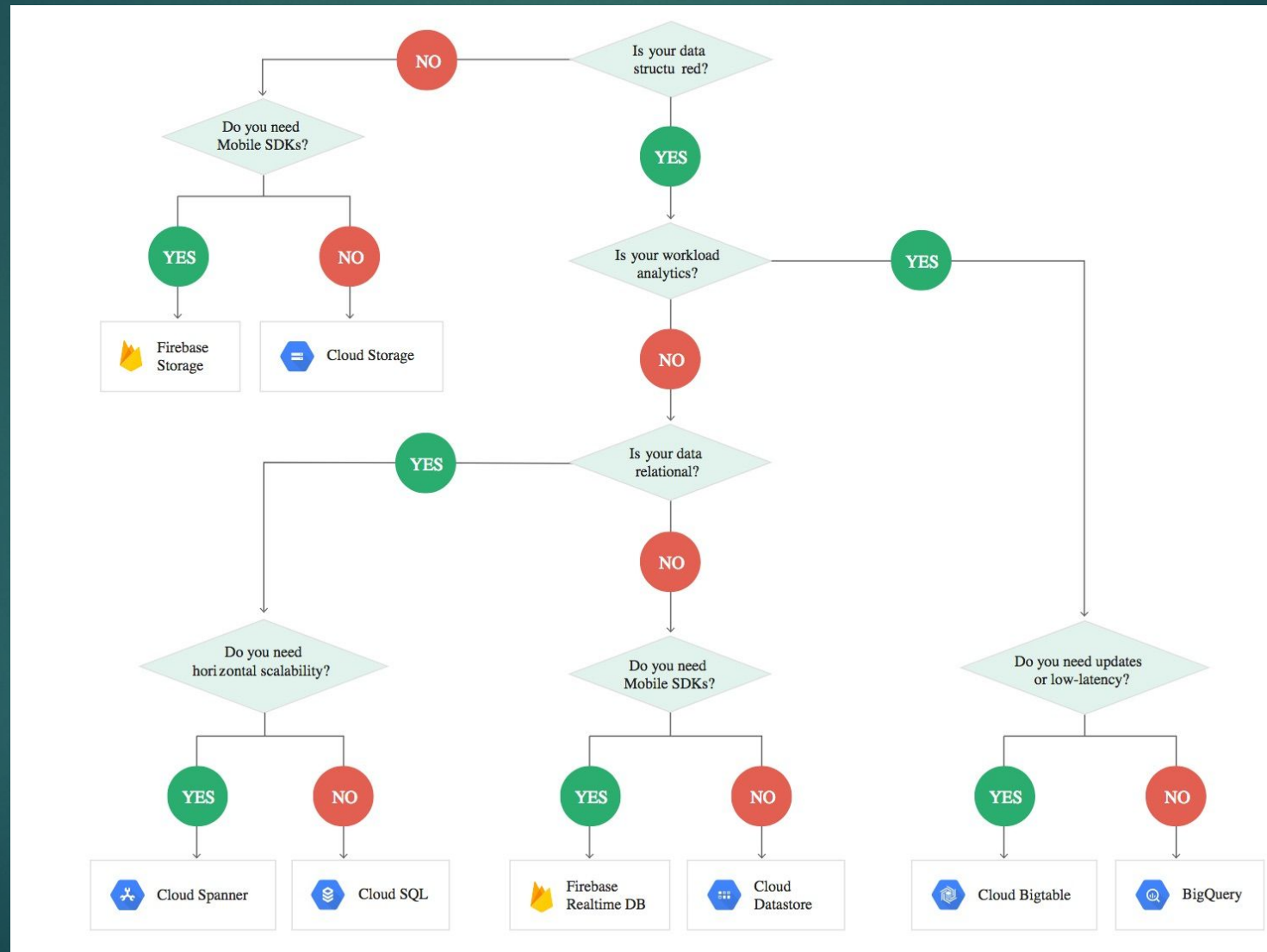
- ▶ NoSQL
 - ▶ Scalable horizontalement
 - ▶ Non relationnel
 - ▶ Attention aux redondances de données
 - ▶ Maîtriser son schéma de document
 - ▶ Versioning
 - ▶ Descriptif du schéma

Compétences nécessaires

- Consistency, availability & partition tolerance tradeoff



Compétences nécessaires



Compétences nécessaires

- ▶ In-memory databases
 - ▶ Avantages
 - ▶ Micro-second read latency & milli-second write latency
 - ▶ High throughput handling
 - ▶ High scalability
 - ▶ Utilisation
 - ▶ Rate limitation
 - ▶ Caching
 - ▶ Concurrency handling
 - ▶ Désavantages
 - ▶ Non relationnel
 - ▶ Aucune persistance de la donnée

Compétences nécessaires

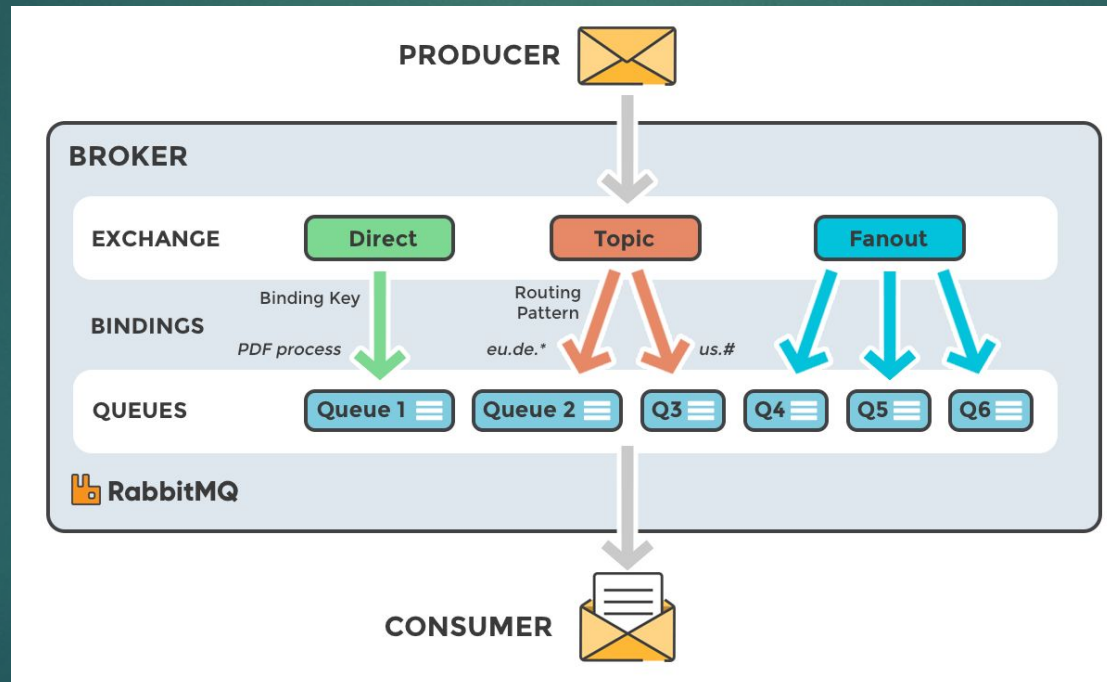
- ▶ Data warehouse / data mining
 - ▶ Stockage de toutes les events dénormalisés
 - ▶ Fonctionne avec du SQL
 - ▶ Utilisation d'outils de BI comme Metabase, Looker, etc...
 - ▶ Schéma doit être très réfléchi et "généreux" dès le départ
 - ▶ Données horodatées (time series data)

Compétences nécessaires

- ▶ Monitoring
 - ▶ Outils utiles
 - ▶ Datadog
 - ▶ Stackdriver
 - ▶ Grafana
 - ▶ Dynatrace
 - ▶ Alerting
 - ▶ Downtime
 - ▶ Error in code
 - ▶ Security/Audit logs
 - ▶ On call engineers

Compétences nécessaires

► Queuing



Compétences nécessaires

- ▶ Queuing
 - ▶ Retry logic
 - ▶ Gestion des doublons
 - ▶ Risque de concurrence
 - ▶ Utile pour alléger vos API
 - ▶ Nécessité d'acknowledge un message lorsqu'il est traité
 - ▶ Deadline pour traitement permet de gérer les processus bloqués

Compétences nécessaires

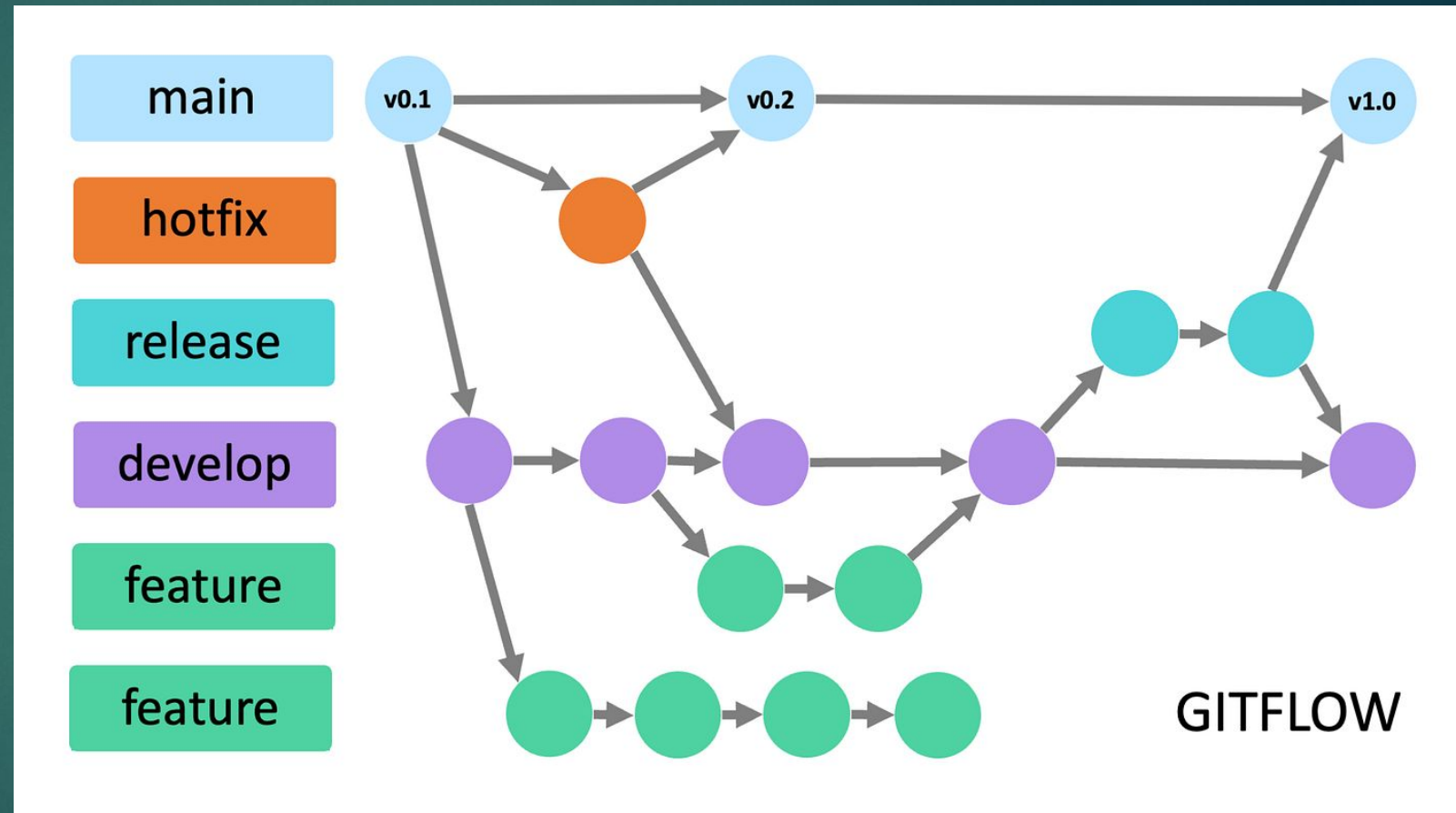
- ▶ Devops / sysadmin
 - ▶ Hébergement sur serveurs dédiés
 - OU
 - ▶ Cloud solution with VMs (Amazon (AWS), Azure, Google Cloud)
 - ▶ Autres hosting, heroku, digital ocean, hostinger, render, vercel
- ▶ Continuous delivery/integration
 - ▶ Jenkins
 - ▶ Github actions/Pipelines (Bitbucket)/Gitlab
 - ▶ Google cloud build
- ▶ Git flow

Compétences nécessaires

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking
You manage Service provider manages			

Compétences nécessaires

- ▶ Git flow CLI
 - ▶ git flow XXX
 - ▶ hotfix start
 - ▶ hotfix finish
 - ▶ feature start
 - ▶ feature finish
 - ▶ release start
 - ▶ release finish



- ▶ Trunk based development

Compétences nécessaires

- ▶ Formation
 - ▶ Faire participer vos équipes à des séminaires
 - ▶ Former vos équipes en interne
 - ▶ Réduction de temps de code review
 - ▶ Réduction du temps d'onboarding
 - ▶ Mettre en place des reviews à l'intérieur de l'équipe

Compétences nécessaires

- ▶ Management
 - ▶ Équilibre nécessaire entre process et agilité
 - ▶ Pair programming
 - ▶ R&D
 - ▶ Gérer les priorités et donner de la visibilité sur ces priorités
 - ▶ Pas un gourou, pas une nounou mais + un coach
 - ▶ Rapport d'étonnements
 - ▶ Gestion du support
 - ▶ Résumer les informations importantes
 - ▶ Insérer dans le backlog les items "enhancement"
 - ▶ Prévoir de la place pour les éléments de votre team gérant les bugs à fix

Compétences nécessaires

- ▶ Les contraintes et avantages du remote work
 - ▶ Difficultés pour avoir de la visibilité sur le travail en cours
 - ▶ Nécessité de manager à la tâche plutôt qu'au travail
 - ▶ Problèmes pour la R&D et pour les tâches difficilement mesurables
 - ▶ Team building et esprit d'équipe
 - ▶ Concentration et qualité des productions
 - ▶ Attendu et demandé par les employés

Compétences nécessaires

- ▶ Management
 - ▶ Management par les faits
 - ▶ Ubiquité
 - ▶ Github projects
 - ▶ Automations
- ▶ Mettre en place des objectifs SMART
 - ▶ Spécifique
 - ▶ Mesurables
 - ▶ Atteignables
 - ▶ Réalistes
 - ▶ Temporel

Compétences nécessaires

- ▶ Management
 - ▶ Définition d'un backlog
 - ▶ Granularité des tâches dans un projet doit être limitée à max 1 jour
 - ▶ Diagrammes de gantt ne sont plus utilisés, raisons:
 - ▶ Flexibilité
 - ▶ Itérations rapide

Compétences nécessaires

- ▶ Management
 - ▶ Comment estimer la durée d'une tâche
 - ▶ Ancienneté des ressources (employés)
 - ▶ Familiarité des concepts pour les personnes
 - ▶ Risques de la tâche (et donc temps nécessaire au testing)
 - ▶ Les performances n'évoluent pas de manière proportionnelle avec le nombre de personnes impliquées sur la tâche (loi des rendements décroissants)

Recrutement

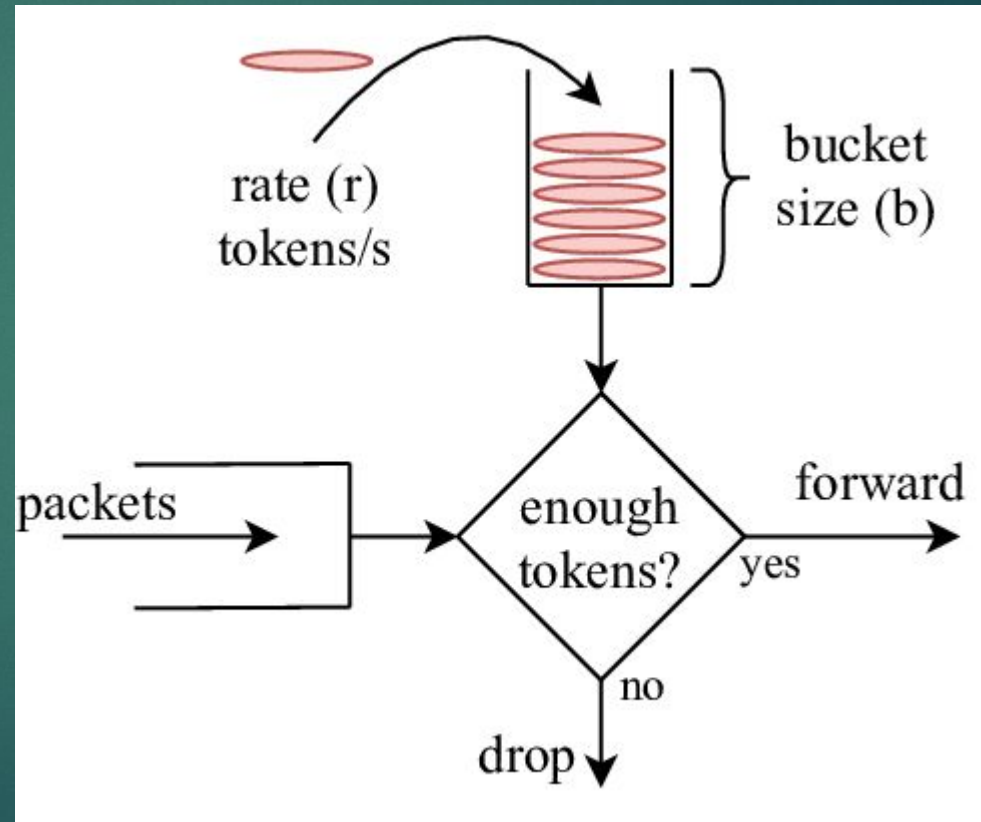
- ▶ Tests techniques adapté
 - ▶ Non reliés de manière fonctionnelle à votre application
 - ▶ (pourrait être considéré comme du travail gratuit)
 - ▶ Relié de manière technique à votre entreprise (même stack)
 - ▶ Tester l'adaptation d'un développeur
 - ▶ Représente les lettres de motivation d'antan
 - ▶ Le CV ne fait pas le développeur
- ▶ Détecter les anomalies lors de la période d'essai (le plus tôt est le mieux)

Communication

- ▶ En contact direct avec les autres départements
- ▶ Travail de vulgarisation
 - ▶ Exercice, essayez d'expliquer le bug qui vous bloque/la contrainte que vous avez à quelqu'un de non tech et obtenir un "je comprend" qui ne veuille pas dire "tais-toi"
- ▶ Faire interface pour votre dev team
- ▶ Estimer un backlog en fonction de la vélocité de votre team

Divers

- ▶ Rate limitation
 - ▶ In-Memory database
 - ▶ Caching
 - ▶ Token bucket algorithm



Divers

- ▶ Http verbs
 - ▶ Idempotency

Verb	Path	Action	Used for
GET	/photos	index	Display list of all photos
POST	/photos	create	Create a new photo
GET	/photos/:id	show	Display a specific photo
PUT	/photos/:id	update	Update a specific photo
DELETE	/photos/:id	destroy	Delete a specific photo

Docker

- ▶ Donne la possibilité de packager une application (Docker image)
- ▶ Virtualise et permet un environnement d'exécution commun
- ▶ Docker container: instance exécutée d'une image
- ▶ Une image docker est décrite par un fichier Dockerfile
- ▶ Elle est hébergée sur un container registry
- ▶ 3 phases:
 - ▶ Build
 - ▶ Push
 - ▶ Execution

Docker (Imperative)

.Dockerfile

```
docker buildx build -t TAG .
```

```
docker run -p 8080:8080 TAG .
```

```
FROM node:16
```

```
WORKDIR /usr/src/app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 8080
```

```
CMD [ "npm", "start" ]
```


Docker (Declarative)

compose.yaml

docker compose up

```
services:

  app:

    image: node:18-alpine

    command: sh -c "yarn install && yarn run dev"

    ports:

      - 127.0.0.1:3000:3000

    working_dir: /app

    volumes:

      - ./:/app

    environment:

      MYSQL_HOST: mysql

      MYSQL_USER: root

      MYSQL_PASSWORD: secret

      MYSQL_DB: todos
```

To Add

- ▶ Service accounts
- ▶ Restful API
- ▶ Stateless API
- ▶ Conventions de codes